MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (HONOURS) REGULAR EXAMINATION — March 2026

Fourth Semester — MG4DSCCSC202

INTRODUCTION TO DATABASE MANAGEMENT SYSTEMS

(2024 Admission Onwards)

Duration: 1 Hour 30 Minutes Maximum Marks: 50

Note: Credit should be given for any correct equivalent answer. Marks may be awarded proportionally based on key points covered. This is a Minor course — flexible evaluation is encouraged.

PART A — Very Short Answer Questions (Answer All) 2 × 5 = 10 Marks

1. Define a field in a DBMS. **(2 marks)**

Answer:

- A field is the smallest meaningful unit of data in a database.
- It stores one specific attribute of a record. e.g., Name, Age, Roll Number in a student record.

(1 mark for definition + 1 mark for example)

2. Explain one-to-one relationship with an example. **(2 marks)**

Answer:

- Each record in Table A is linked to exactly one record in Table B, and vice versa.
- Example: One person has one passport; one student has one ID card.
- Implemented using a Foreign Key with a UNIQUE constraint.

(1 mark for definition + 1 mark for example)

3. Differentiate between a single line and a dashed line in an ER diagram. **(2 marks)**

Answer:

- **Single line (solid line):** Represents partial participation — not all entity instances must participate in the relationship.
- **Double line (two parallel lines):** Represents total participation — every entity instance must participate.
- **Dashed ellipse:** Represents a derived attribute (e.g., Age derived from Date of Birth).
- **Dashed underline:** Represents the discriminator (partial key) of a weak entity.
- Note: The question contrasts 'single line' with 'dashed line' — single line = partial participation; dashed elements = derived attribute or discriminator.

(1 mark for single line = partial participation + 1 mark for dashed line/element with correct meaning)

4. Name the components of a MySQL table. **(2 marks)**

Answer:

- Columns (Fields) — store individual attributes
- Rows (Records / Tuples) — one complete entry
- Primary Key — uniquely identifies each row
- Data Types — define kind of data per column (INT, VARCHAR, DATE, etc.)
- Constraints — rules like NOT NULL, UNIQUE, DEFAULT, FOREIGN KEY

(Any 4 components — ½ mark each)

5. List any four privileges that can be granted in MySQL. **(2 marks)**

Answer:

- SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALL PRIVILEGES

(Any 4 correct privileges — ½ mark each)

PART B — Short Answer Questions (Answer Any 5) 5 × 5 = 25 Marks

6. Explain how DBMS reduces data redundancy. **(5 marks)**

Answer:

• Data redundancy = same data stored in multiple places, causing waste and inconsistency.
• DBMS reduces redundancy through:
• **Centralised storage:** All data stored once; all applications access the same source.
• **Normalisation:** Tables are organised so each fact is stored exactly once.
• **Foreign Keys:** Tables reference each other instead of repeating data (e.g., DeptID in EMPLOYEE table instead of full department details).
• **Data Dictionary:** DBMS tracks all stored data, preventing accidental duplication.
• Example: Student phone number stored once in STUDENT table; other tables refer by student ID only.

(1 mark for defining redundancy + 1 mark each for 4 methods)

7. Differentiate between data and information with examples. **(5 marks)**

Answer:

• **Data:** Raw, unprocessed facts with no meaning on their own.
• **Information:** Processed, organised data that is meaningful and useful for decision-making.
• Differences:
• Data is input; information is output of processing.
• Data has no context; information has meaning and purpose.
• Data alone is not useful; information supports decisions.
• Examples:
• Data: 85, 70, 90 → Information: Marks of three students in Chemistry.
• Data: 9846000000 → Information: Mobile number of the lab assistant.
• Data: Monday, 38, 12 → Information: On Monday, 38 attended class; 12 were late.

(2 marks for definitions + 3 marks for differences and examples)

8. Explain the role of end users with examples. **(5 marks)**

Answer:

• End users are people who interact with the database regularly for their work tasks.
• Types:
• **Naive / Parametric Users:** Perform routine pre-defined tasks via ready-made applications. e.g., bank cashier processing transactions.
• **Casual / Ad-hoc Users:** Access database occasionally; may form their own queries. e.g., manager running monthly attendance report.
• **Sophisticated Users:** Write complex queries; familiar with database tools. e.g., data analyst or research scientist.
• **Standalone Users:** Maintain personal small databases using desktop software. e.g., faculty member tracking marks in a personal database.

(1 mark for definition + 1 mark each for any 4 types with examples)

9. Describe how referential integrity ensures the connection between two related tables. **(5 marks)**

Answer:

• Referential integrity ensures that a Foreign Key (FK) value in a child table always matches a valid Primary Key (PK) value in the parent table, or is NULL.
• Rules enforced:
• **Insertion rule:** A record cannot be added to the child table with a FK value that does not exist in the parent table.
• **Deletion rule:** A parent record cannot be deleted if child records still reference it.
• **Update rule:** A PK value in the parent table cannot be changed if child records still refer to it (unless CASCADE is set).
• Example: In a library DB, LOAN.MemberID (FK) references MEMBER.MemberID (PK). A loan cannot be added for a non-existent member.

(2 marks for definition + 3 marks for rules/example)

10. Describe a scenario where partial dependency and transitive dependency occurs. **(5 marks)**

Answer:

• Table: ENROLMENT (StudentID, CourseID, StudentName, CourseName, Marks, DeptID, DeptName)
• Primary Key: {StudentID, CourseID}

Partial Dependency:

• StudentName depends only on StudentID (part of PK) — not on the full composite key.
• CourseName depends only on CourseID (part of PK) — not on the full composite key.
• Non-key attribute depends on only part of the composite primary key.

Transitive Dependency:

• DeptName depends on DeptID, and DeptID depends on StudentID.
• DeptName is not directly dependent on the primary key — it goes through a non-key attribute (DeptID).
• Chain: {StudentID, CourseID} → DeptID → DeptName
• Fix: Decompose into STUDENT, COURSE, ENROLMENT (StudentID, CourseID, Marks), DEPARTMENT tables.

(2.5 marks for partial dependency with example + 2.5 marks for transitive dependency with example)

11. Explain the difference between COUNT(*), COUNT(column), and COUNT(DISTINCT column) with examples. **(5 marks)**

Answer:

• **COUNT(*):** Counts all rows including rows with NULL values.

```
SELECT COUNT(*) FROM EMPLOYEE;   -- returns total rows e.g. 10
```

• **COUNT(column):** Counts only non-NULL values in the specified column.

```
SELECT COUNT(Department) FROM EMPLOYEE;   -- returns 8 if 2 rows have NULL dept
```

• **COUNT(DISTINCT column):** Counts only unique non-NULL values; duplicates counted once.

```
SELECT COUNT(DISTINCT Department) FROM EMPLOYEE;   -- returns 3 if depts are HR, IT, Finance
```

(1 mark each for definition + example of all three functions = 3 marks; 1 mark for contrast; 1 mark for clarity)

12. Explain the UPDATE statement in MySQL. Demonstrate how to update a single column, multiple columns, and update based on a condition with examples. **(5 marks)**

Answer:

• UPDATE modifies existing records in a table.
• Syntax: UPDATE table_name SET column = value WHERE condition;
• Without WHERE, all rows are updated — usually unintended.

(a) Update single column:

```
UPDATE EMPLOYEE SET Salary = 35000 WHERE EmpID = 101;
```

(b) Update multiple columns:

```
UPDATE EMPLOYEE SET Salary = 40000, Department = 'HR' WHERE EmpID = 102;
```

(c) Update based on condition (multiple rows):

```
UPDATE EMPLOYEE SET Salary = Salary * 1.10 WHERE Department = 'IT';
```

(1 mark for syntax + 1 mark each for 3 scenarios + 1 mark for WHERE clause note)

PART C — Essay Questions (Answer Any 1) 15 × 1 = 15 Marks

13. Explain the relational data model in detail. Describe domain, attribute, tuple, and relation with examples. **(15 marks)**

Answer:

• The relational data model (E.F. Codd, 1970) organises data as two-dimensional tables called relations.

1. Domain [4 marks]

• The set of all permitted atomic values for an attribute.
• Defines the data type and valid range for a column.
• Examples: Domain of Age = positive integers 1–120; Domain of Department = {Science, Arts, Commerce}.

2. Attribute [4 marks]

• A named column in a relation, associated with a specific domain.

• Describes one property of the entity being stored.

• Example: STUDENT table has attributes StudentID, Name, Age, Department. Degree = number of attributes.

3. Tuple [4 marks]

• A single row in a relation representing one real-world entity instance.

• Contains exactly one value per attribute.

• Example: (101, 'Arjun Menon', 20, 'Science') is one tuple in STUDENT. Cardinality = number of tuples.

4. Relation [3 marks]

• A complete two-dimensional table with a schema (structure) and an instance (current data).

• Properties: all cell values are atomic; column names are unique; no duplicate rows; row/column order does not matter.

• Example: STUDENT (StudentID, Name, Age, Department) — schema; all current student rows — instance.

14. Using EMPLOYEE(EmpID, Name, Department, Salary), write SQL queries for: (a) WHERE, (b) ORDER BY, (c) GROUP BY, (d) HAVING, (e) LIKE, (f) BETWEEN. Explain the purpose of each clause. **(15 marks)**

Answer:

• Table: EMPLOYEE (EmpID, Name, Department, Salary)

(a) WHERE [2.5 marks]

• Purpose: Filters rows based on a condition; only matching rows are returned.

```
SELECT * FROM EMPLOYEE WHERE Salary > 30000;
```

• Returns all employees earning more than 30,000.

(b) ORDER BY [2.5 marks]

• Purpose: Sorts the result set. Default is ASC (ascending); use DESC for descending.

```
SELECT * FROM EMPLOYEE ORDER BY Name ASC;
```

• Returns all employees sorted alphabetically by name.

(c) GROUP BY [2.5 marks]

• Purpose: Groups rows with the same column value. Used with aggregate functions like COUNT, SUM, AVG.

```
SELECT Department, COUNT(*) AS EmpCount FROM EMPLOYEE GROUP BY Department;
```

• Returns count of employees in each department.

(d) HAVING [2.5 marks]

• Purpose: Filters groups formed by GROUP BY. Like WHERE but applied after grouping.

```
SELECT Department, COUNT(*) AS EmpCount FROM EMPLOYEE
GROUP BY Department HAVING COUNT(*) > 2;
```

• Returns only departments with more than 2 employees.

(e) LIKE [2.5 marks]

• Purpose: Pattern matching on text columns. % = any number of characters; _ = exactly one character.

```
SELECT * FROM EMPLOYEE WHERE Name LIKE 'A%';
```

• Returns all employees whose name starts with 'A' (e.g., Arjun, Amal, Athira).

(f) BETWEEN [2.5 marks]

• Purpose: Selects rows where a value falls within an inclusive range.

```
SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 20000 AND 50000;
```

• Returns employees with salary from 20,000 to 50,000 (both values included).

(2.5 marks per clause: 1 mark purpose + 1 mark correct query + 0.5 mark output explanation)

Sumaja Sasidharan

Asst.Professor

Sree Sankara Vidyapeetom College

Valayanchirangara